

Maven Project To Connect To Web Servers Using Selenium Dependency

Maven Project to Connect to Web Servers Using Selenium Dependency with respect to Automation Exercise Website

A Maven project using Selenium dependency can be used to automate testing on the [Automation Exercise Website](#). The project helps in connecting to the website, performing automated browser actions, and testing different functionalities of the web application.

In this project, Apache Maven manages Selenium libraries and project dependencies through the pom.xml file. Selenium WebDriver is used to launch the browser, access the Automation Exercise website, interact with webpage elements, and verify outputs automatically.

Purpose of the Project

The main purpose of this project is to perform automated testing on the Automation Exercise website instead of manual testing. It is commonly used for:

- Login testing
- Registration testing
- Product search testing
- Form validation
- Navigation testing
- Regression testing

Working Process

1. Maven downloads Selenium dependencies automatically.
2. Selenium WebDriver launches the browser.
3. The browser connects to the Automation Exercise website.
4. Automated test cases are executed on web pages.
5. Results are verified and displayed.

Advantages

- Reduces manual testing effort
- Faster and accurate testing
- Easy dependency management using Maven
- Supports CI/CD integration with tools like Jenkins
- Improves software testing efficiency and reliability

This project demonstrates how Maven and Selenium work together to automate testing operations on the Automation Exercise website in DevOps and software testing environments.

Steps to execute the project:

Step1:

Create a maven project on the terminal with the command:

```
$mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp03 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```

vaishnav@vaishnavi-VirtualBox: ~$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenSeleniumApp03 -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] ----- org.apache.maven:standalone-pom -----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
>>> mvn archetype-plugin:1.4.2:generate (default-cli) > generate-sources @ standalone-pom >>>
<<< mvn archetype-plugin:1.4.2:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
--- mvn archetype-plugin:1.4.2:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (18 KB at 7.3 MB/s)
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
Parameter: basedir, Value: /home/vaishnavi/devops21
Parameter: package, Value: com.example
Parameter: groupId, Value: com.example
Parameter: artifactId, Value: MyMavenSeleniumApp03
Parameter: packageName, Value: com.example
Parameter: version, Value: 1.0-SNAPSHOT
[INFO] Project created from Old (1.x) Archetype in dir: /home/vaishnavi/devops21/MyMavenSeleniumApp03
[INFO]
BUILD SUCCESS
[INFO]
Total time: 4.45s
[INFO]
Finished at: 2025-03-17T08:58:39+05:30
[INFO]
vaishnav@vaishnavi-VirtualBox: ~$ cd MyMavenSeleniumApp03
vaishnav@vaishnavi-VirtualBox: ~$ mvn archetype:generate
pom.xml

```

The command can be broken down as:

- DgroupId=com.example – project’s base package (reverse-domain style).
- DartifactId=MyMavenSeleniumApp03 – defines the name of the project (will also be the folder name).
- DarchetypeArtifactId=maven-archetype-quickstart – sets the Maven template to be the basic Java application.
- DinteractiveMode=false – deactivates interactive mode and uses the given values automatically.

Step 2:

Use \$cd command to go into the directory.

\$gedit pom.xml

pom.xml file is created to configure the Maven project. It defines the project information, adds required dependencies like Guava and Commons IO libraries, and includes build plugins to compile the Java code, run unit tests, and package the application into an executable JAR file.

<dependencies>

<dependency>

<groupId>junit</groupId>

<artifactId>junit</artifactId>

<version>3.8.1</version>

<scope>test</scope>

</dependency>

<!-- https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->

<dependency>

<groupId>org.seleniumhq.selenium</groupId>

```
<artifactId>selenium-java</artifactId>

<version>4.29.0</version>
</dependency>
</dependencies>

<!-- Build: Configuring plugins and build settings -->
<build>
<plugins>
<!-- Example: Maven Compiler Plugin to compile Java code -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
<source>1.7</source>
<target>1.7</target>
</configuration>
</plugin>
<!-- Example: Maven Surefire Plugin to run tests -->
<plugin>
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-surefire-plugin</artifactId>
<version>2.22.2</version>
</plugin>
<plugin>
    <groupId>org.apache.maven.plugins</groupId>

    <artifactId>maven-jar-plugin</artifactId>
    <version>3.0.1</version>
    <configuration>
        <archive>
            <manifestEntries>
                <Main-Class>com.example.App</Main-Class>
            </manifestEntries>
        </archive>
```

```
</configuration>
```

```
</plugin>
```

```
</plugins>
```

```
</build>
```

```
</project>
```

Step 3:

inspect the AutomationExercise webpage to find the element IDs required for automation.

Right-click on the webpage and select Inspect. The HTML code will be displayed. Click on the cursor icon (arrow symbol) available on the top-left of the inspect panel, then hover over the username field to identify its ID, which is user-name.

Similarly, check the search button (ID:submit_search). Make a note of these IDs, as they will be used in the App.java file.

Next,

Edit the App.java file

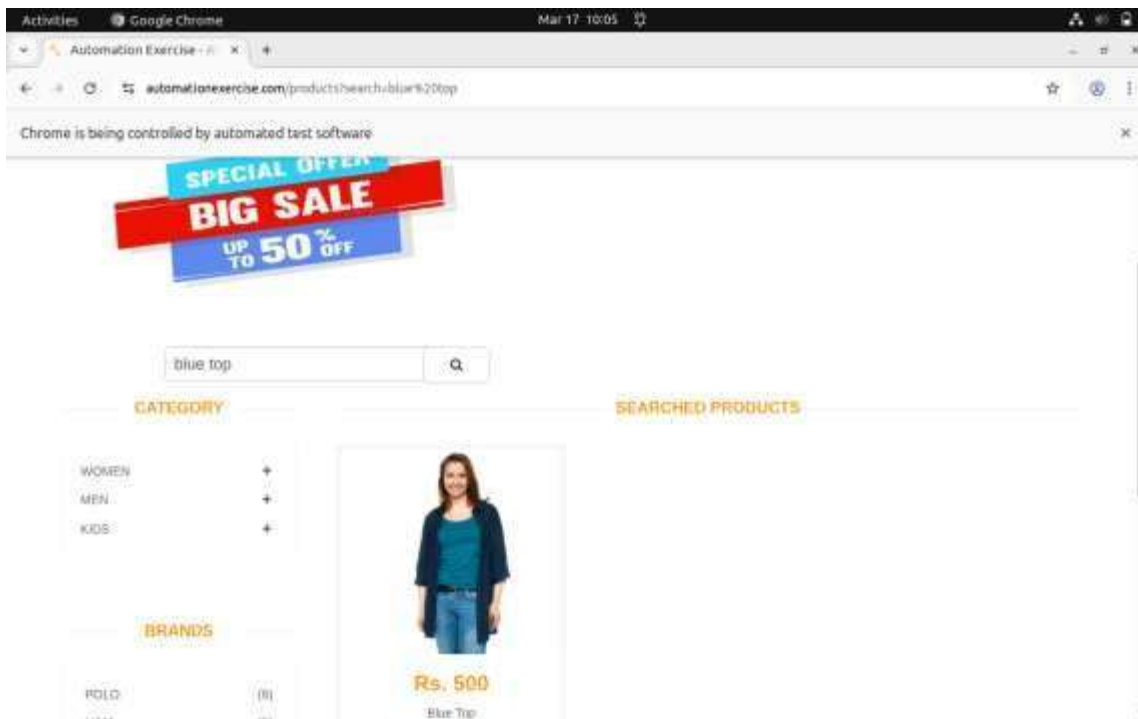
```
package com.example;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

/** * Hello world! * */ public
class App
{
    public static void main( String[] args )
    {
        WebDriver driver=new ChromeDriver();
        driver.get("https://automationexercise.com/products");
        driver.manage().window().maximize();
        driver.findElement(By.id("search_product")).sendKeys("blue top");
        driver.findElement(By.id("submit_search")).click();
    }
}
```

Step 4:

Build and run the maven application



Automation Exercise Website

<https://automationexercise.com/products> is a sample website used for testing and learning web automation tools like Selenium.

It is used for:

- Writing automation scripts and UI tests
- Practicing login functionality testing
- Learning and improving test automation skills

Pushing The Project into Git:

Execute the commands:

```
$ git config --global user.name "vaishnavim1121"
```

```
$ git config --global user.email "vaishnavim1121@gmail.com"
```

```
$ git init
```

```
$ git add .
```

```
$ git commit -m "initial commit"
```

```
$ git remote add origin https://github.com/vaishnavim1121/MyMavenSeleniumApp.git
```

```
$ git remote set-url origin https://*PAT*@github.com/vaishnavim1121/MyMavenSeleniumApp
```

```
$ git push -u origin master
```

```

[dev] Total time: 22.289 s
[dev] Finished at: 2026-01-04T22:25:15+05:30
[dev] -----
vaishnav@vaishnavi-VirtualBox: ~/devops1/MyHavenSeleniumApp02 $ git config --global user.name "vaishnavi121"
vaishnav@vaishnavi-VirtualBox: ~/devops1/MyHavenSeleniumApp02 $ git config --global user.email "vaishnavi121@gmail.com"
vaishnav@vaishnavi-VirtualBox: ~/devops1/MyHavenSeleniumApp02 $ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/vaishnavi/devops1/MyHavenSeleniumApp02/.git/
vaishnav@vaishnavi-VirtualBox: ~/devops1/MyHavenSeleniumApp02 $ git add .
vaishnav@vaishnavi-VirtualBox: ~/devops1/MyHavenSeleniumApp02 $ git commit -m "Initial"
[master (root-commit) 7230bfc] Initial
13 files changed, 204 insertions(+)
create mode 100644 pom.xml
create mode 100644 src/main/java/com/example/App.java
create mode 100644 src/test/java/com/example/AppTest.java
create mode 100644 target/MyHavenSeleniumApp02-1.0-SNAPSHOT.jar
create mode 100644 target/classes/com/example/App.class
create mode 100644 target/maven-archiver/pom.properties
create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/createdFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/compile/default-compile/inputFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/createdFiles.lst
create mode 100644 target/maven-status/maven-compiler-plugin/testCompile/default-testCompile/inputFiles.lst
create mode 100644 target/surefire-reports/TEST-com.example.AppTest.xml
create mode 100644 target/surefire-reports/com.example.AppTest.txt
create mode 100644 target/test-classes/com/example/AppTest.class
vaishnav@vaishnavi-VirtualBox: ~/devops1/MyHavenSeleniumApp02 $ git remote add origin https://github.com/vaishnavi121/MyHavenSeleniumApp02.git
vaishnav@vaishnavi-VirtualBox: ~/devops1/MyHavenSeleniumApp02 $ git remote set-url origin https://ghp_WwyMq05gt2QwFw7mcJtJalSZv3w4IFI@github.com/vaishnavi121/MyHavenSeleniumApp02
vaishnav@vaishnavi-VirtualBox: ~/devops1/MyHavenSeleniumApp02 $ git push -u origin master
Enumerating objects: 39, done.
Counting objects: 100% (39/39), done.
Delta compression using up to 16 threads
Compressing objects: 100% (19/19), done.
Writing objects: 100% (39/39), 8.51 KiB | 1.21 MiB/s, done.

```

Second Phase

Depicting Jenkins Pipeline

Follow the given steps to deploy the project on Jenkins.

create a Jenkins pipeline job that will use the Jenkinsfile from your project repository

go to the Jenkins home page and click on “New Item” to create a job, enter the item name select Pipeline as the project type.

▼ / New Item

New Item

Enter an item name

Select an item type

-  **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
-  **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
-  **Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
-  **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click “OK” to proceed to the job configuration page

Definition

Pipeline script from SCM

SCM

Git

Repositories

Repository URL

<https://github.com/vaishnavim1121/SeleniumAutomation.git>

Please enter Git repository.

Credentials

github credentials

+ Add

Advanced

Save Apply

Now we need to add a jenkinsfile into our project which will consist of the operations that will occur when building the project in Jenkins

It includes step wise instructions guiding at each stage with right command

```
pipeline {
    agent any // use any available agent

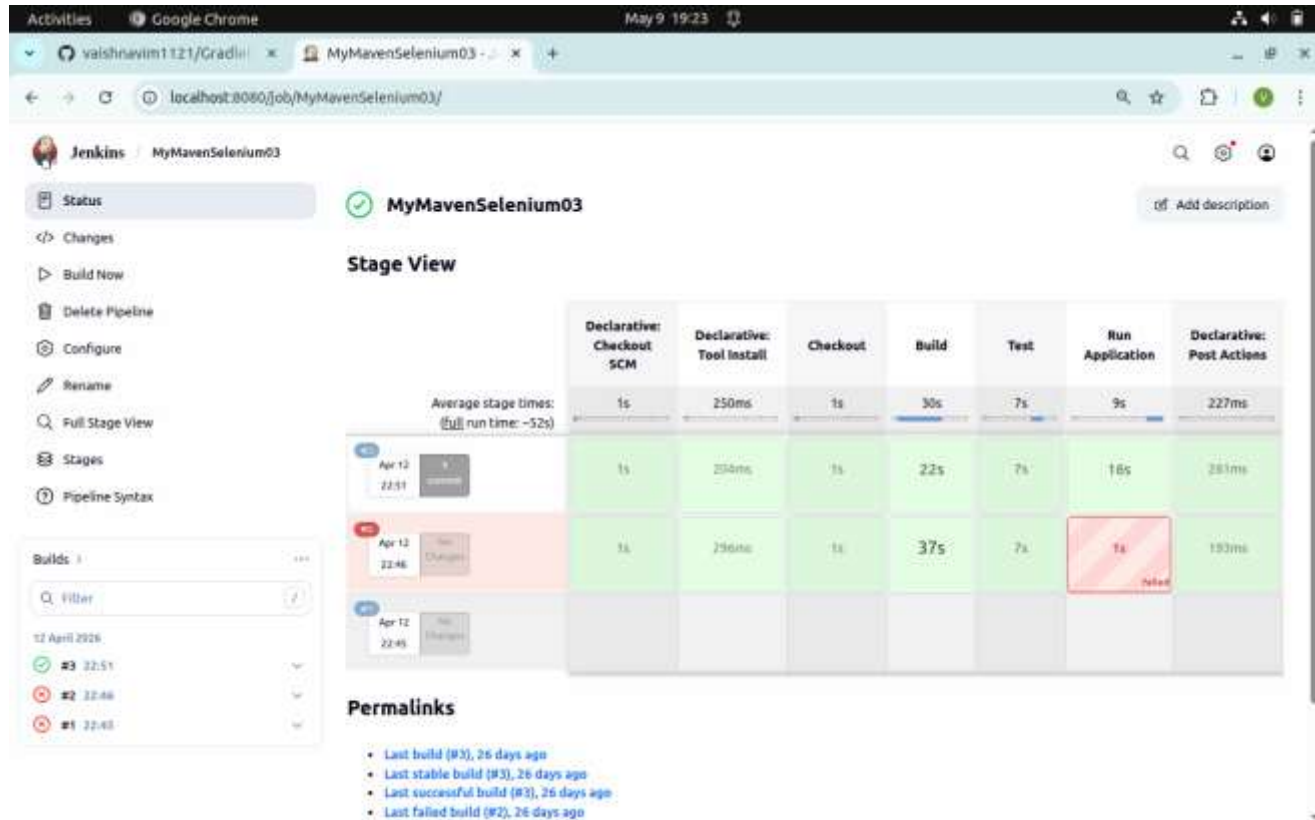
    tools {
        maven 'Maven' // Ensure this matches the name configured in Jenkins
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'master', url: 'https://github.com/vaishnavim1121/MyMavenSeleniumApp01.git'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn clean package' // Run Maven build
            }
        }

        stage('Test') {
            steps {
                sh 'mvn test' // Run unit tests
            }
        }

        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenSeleniumApp01-1.0-SNAPSHOT.jar'
            }
        }
    }
}
```


Now try building the project with Jenkins



The error was due to failure in github repository, by directing the correct link to the Jenkins, and re-building it, the project is built successfully,